

Module 11

Maintain a secure repository
by using GitHub best practices



Agenda



GitHub Foundations Part 1

- 1 Introduction to Git
- 2 Introduction to GitHub
- 3 Introduction to GitHub's products
- 4 Configure code scanning on GitHub
- 5 Introduction to GitHub Copilot
- 6 Code with GitHub Codespaces
- 7 Manage your work with GitHub Projects
- 8 Communicate effectively on GitHub using Markdown

GitHub Foundations Part 2

- 9 Contribute to an open-source project on GitHub
- 10 Manage an InnerSource program by using GitHub
- 11 Maintain a secure repository by using GitHub best practices
- 12 Introduction to GitHub administration
- 13 Authenticate and authorize user identities on GitHub
- 14 Manage repository changes by using pull requests on GitHub
- 15 Search and organize repository history by using GitHub
- 16 Using GitHub Copilot with Python

Introduction

GitHub offers various automated features that allow you to efficiently administer a secure repository without slowing everyone down throughout the entire development process.

By the end of this module, you'll be able to:

- Recognize the importance of securing your repository and shifting left in the development lifecycle.
- Identify the tools, GitHub features, and best practices to establish a secure development strategy.
- Keep sensitive files out of your repository by applying the use of a .gitignore file.
- Identify how to detect and fix outdated dependencies with security vulnerabilities.
- Recognize advanced security features such as code scanning and secret scanning.

How to maintain a secure GitHub repository



Benefits of DevOps Practices

- **Integration of Security Testing:** DevOps practices make it easier to integrate security testing into the pipeline, making it a part of daily processes.
- **Developer-Centric Security:** Shifting left focuses on making security more developer-centric, providing feedback early and reducing overall development time.

Importance of Integrating Security Early

- **Secure Development Lifecycle:** Security must be integrated at every stage of the software development lifecycle.
- **Shift Left:** Moving security processes earlier in the development lifecycle reduces mistakes and speeds up development.

GitHub Security Advisories

- **Private Collaboration:** Maintainers can privately discuss and fix security vulnerabilities before publicly disclosing them.
- **Community Updates:** Published advisories help the community update dependencies and understand the impact of vulnerabilities.

How to maintain a secure GitHub repository (cont.)

Communicate a Security Policy with SECURITY.md

- Provides guidance for responsible disclosure of security issues, speeding up resolution.
- Place the SECURITY.md file in the root of the repository for easy access.

GitHub Security Advisories

- Private Discussions: Allows maintainers to privately discuss and fix security vulnerabilities.
- Public Disclosure: Maintainers can publish advisories to inform the community.

Keep Sensitive Files Out with .gitignore

- Prevents accidental commits of sensitive data by ignoring specified files and patterns.
- Include common patterns like build results and user-specific files in .gitignore.

.venv/

Remove Sensitive Data from a Repository

- Assume any committed data is compromised; simply overwriting commits is not enough.
- Follow GitHub's guide to thoroughly remove sensitive data from the repository.

Branch Protection Rules

- Create rules to enforce workflows, such as requiring reviews or passing status checks.
- Use rules to run builds, linters, and automated tests to ensure code quality.

Define Workflows

- Specify workflows for branches, pull requests, and releases to guide contributors.
- Clearly communicate the workflow strategy to all contributors.

Automated security

- **Dependabot Alerts** monitor dependency graphs and cross-references with known vulnerability lists to alert projects of risks.
- **Input Source** utilizes GitHub Security Advisories for analysis and risk detection.

Dependabot alerts

Configure ▾

is:open

☐  5 Open

☒ 0 Closed

Package ▾

Ecosystem ▾

Manifest ▾

Severity ▾

Sort ▾

☐ 	Command Injection in hot-formula-parser Critical	#1 opened now • Detected in hot-formula-parser (npm) • javascript/package-lock.json
☐ 	Command Injection in lodash High	#5 opened now • Detected in lodash (npm) • javascript/yarn.lock
☐ 	Command Injection in lodash High	#3 opened now • Detected in lodash (npm) • javascript/package-lock.json
☐ 	Regular Expression Denial of Service (ReDoS) in lodash Moderate	#4 opened now • Detected in lodash (npm) • javascript/yarn.lock
☐ 	Regular Expression Denial of Service (ReDoS) in lodash Moderate	#2 opened now • Detected in lodash (npm) • javascript/package-lock.json

Automated security (cont.)

Code QL

- **Code Scanning** helps find, triage, and prioritize fixes for security vulnerabilities and prevents new security issues.
- **CodeQL Integration** allows querying code as data for custom or community-maintained queries.

Security overview

Security policy • Disabled Define how users should report security vulnerabilities for this repository	Set up a security policy
Security advisories • Enabled View or disclose security advisories for this repository	View security advisories
Private vulnerability reporting • Disabled Allow users to privately report potential security vulnerabilities	Enable vulnerability reporting
Dependabot alerts • Enabled Get notified when one of your dependencies has a vulnerability	View Dependabot alerts
Code scanning alerts • Needs setup Automatically detect common vulnerability and coding errors	Set up code scanning
Secret scanning alerts • Enabled Get notified when a secret is pushed to this repository	View detected secrets

Exercise - Secure your repository's supply chain



This exercise is a challenge based on content covered in this module, and there could be more than one way to successfully complete the exercise.

If you get stuck, revisit previous content in this module or navigate to some of the other resources provided.

In this exercise, you will explore:

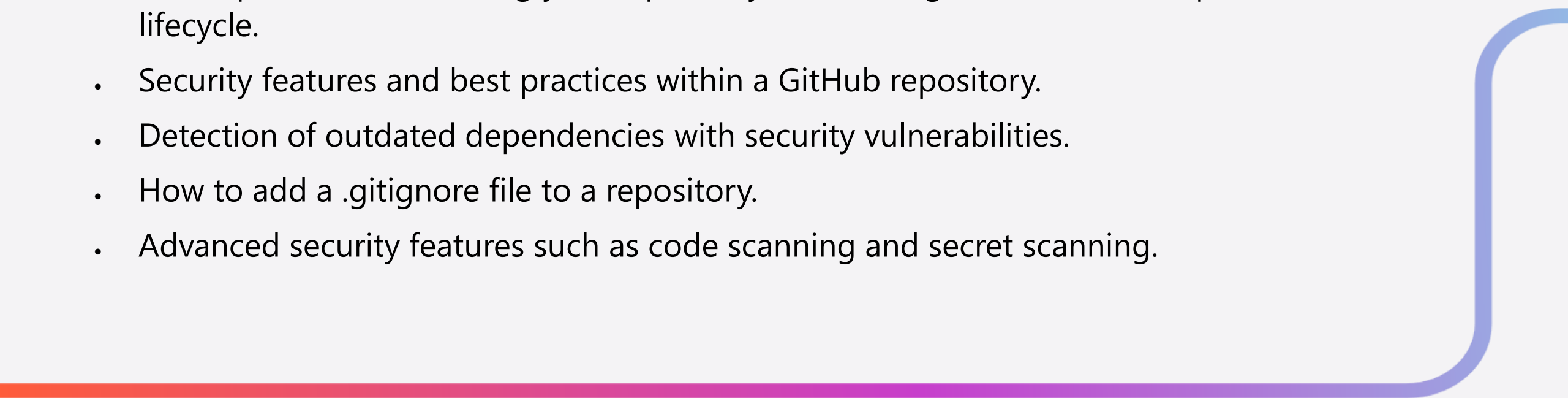
- Dependency graph
- Dependency alerts
- Dependency security updates
- Dependency versions updates

[Start the exercise on GitHub](#)

Summary

In this module, we talked about the importance of securing and maintaining a GitHub repository.

You learned about:

- The importance of securing your repository and shifting left in the development lifecycle.
 - Security features and best practices within a GitHub repository.
 - Detection of outdated dependencies with security vulnerabilities.
 - How to add a .gitignore file to a repository.
 - Advanced security features such as code scanning and secret scanning.
- 

End of presentation



